

VR-CLIP (CVPR 2026 一作)

CLIP在图像级任务上表现很强，但做像素级密集预测时有个本质问题：CLIP 预训练的目标是全局图文对齐（对比学习），ViT 的全局注意力让同一物体内部的 patch 特征不连贯，特征过度集中在最具区分性的区域。现有方法主要靠更强的decoder来弥补，但终究受限于输入特征的质量。

最开始思路是用微调视觉编码器，我们发现这会破坏 CLIP 的视觉-语言对齐能力（**从语义的角度：视觉特征改变了，文本编码器却冻结着，两者原本共享的嵌入空间被拉开**）。所以问题的核心是：如何在保持视觉-语言对齐的前提下，提升像素级判别能力？

NSA：从不影响语义的角度出发捕捉浅层局部信息。浅层编码的是低级模式（边缘、纹理），天然需要局部卷积聚合；深层关注全局语义，卷积帮助有限，且过度修改会破坏预训练表示。 Self-Attention 建的是全局长程依赖，NSA 额外建模的是局部邻域空间关系。NSA 插在 CLIP ViT 的浅层 1-2 layers，把 token 序列展开回 2D 特征图，用深度可分离卷积 + 通道混合，聚合局部邻域信息，增强空间连续性。

SGR：从对齐好的文本向量角度，引入一组语义 tokens，这 100 个 tokens 是可学习的，但不是随机初始化。我们用类别 embedding 去初始化/约束它们，相当于给这些 tokens 一个先验：起点必须在 CLIP 已经建好的语义空间里，训练时不会跑偏到无关的随机方向。这样既保留了 CLIP 的对齐能力，又给了模型足够的自由度去学更细粒度的语义分解。（为什么 100 个？原始类别级向量太粗了，像素级分割需要的语义粒度远不止“这是猫”——它还要区分猫的边缘、纹理、部位关系等，太少的话信息容量不够）

然后基于相似度映射将关联语义信息注入视觉 patch 特征，残差式注入原始路径（为了让语义修正始终是对原始 CLIP 特征的增量微调，而不是覆盖式重写）。

另外我们采用跨层 MLP 参数共享机制，学习共性映射，unseen 急剧提升+5%。SGR 里有两个 MLP，一个做语义聚合，一个生成残差。**共享是为了减少参数，同时强制所有层用同一套映射规则。独立参数会让每层学各自的语义映射，绑定到 seen 类的分布上，过拟合风险大。共享参数强制学习一个跨层统一、与具体类别无关的“共性映射”，新类别的语义 token 也能走同样的投影路径，泛化更稳。**由于所有层共享同一个映射，这个映射在训练时被 12 个不同深度的特征监督，相当于获得了 12 倍的监督信号（虽然数据相同，但输入分布不同），共享也起到了**隐式的正则化作用**。

训练参数量只占 8 %。

1. 这个工作最大的 failure case 是什么？如果现在让你继续做这个方向，你下一步会怎么改进？

极端光照/低对比度场景：昏暗环境下的类别，像素特征本身就很弱，语义修正信号容易被噪声淹没。
小目标：因为 ViT/16 的 patch 粒度本身就是瓶颈，方法层面没有专门处理。倾向于在 encoder 内部把 SAM 的 patch 特征作为辅助引导信号（比如加到 NSA 里做混合局部聚合）。

通用推理增强 (Post-training)

增强大模型推理能力现有方法要么需要人工标注推理过程（成本极高），要么只能处理特定领域。可验证领域：传统 RLVR 方法（如DeepSeek-R1）主要局限在数学和代码领域，因为它们依赖规则验证器——数学题可以验证答案对错，代码可以跑测试用例。不可验证领域：基于奖励模型（reward hacking）。**如何为通用领域推理任务提供有效的奖励信号？**

核心思想是 **概率奖励** —— 用模型对参考答案的解码概率作为奖励信号，相当于奖励的是“这段推理对参考答案的支持程度”，即推理质量本身。要不要举个例子？

优势：RLVR 奖励信号没这个细致（0-1）、奖励模型容易遇到 reward hacking！（GRPO本身、奖励去偏）。序列似然对低概率 token 极度敏感，奖励方差极大，需要限制答案长度 ≤ 7 token，我们突破了长度限制。

整体采用 冷启动 SFT + RLPR 两阶段 方案。冷启动阶段：基于 DeepSeek-R1 范式，用 Few-shot Prompting合成 8 K 高质量 CoT 数据，让模型学会规范的推理格式。

奖励去偏：模型对标准答案的解码概率天然受两个因素影响——推理质量（我们想要的）、题目本身难度（系统偏差）。例如高数题的参考答案概率天然低于常识题，简单题的基线概率本来就高，这会导致奖励不可比。把推理过程去掉，直接问模型“这个问题的答案是什么”的概率，通过减去无推理时的基线概率，消除题目本身难度偏差。

自适应标准差过滤：动态过滤太简单或太难的样本，这些样本梯度信号极弱，浪费计算甚至导致训练不稳定，通过计算这 8 个奖励的标准差 $\sigma < \text{阈值}$ 时。训练初期模型能力弱，随着模型变强，标准差自然提高，阈值也相应上浮。

训练：基于 GRPO 算法，去除 KL 约束（KL_coef=0），提升 clip_eps=0.27，提升探索空间。（**为什么敢去除 KL？GRPO 的组内相对优势已经提供了足够的稳定性约束，同一 prompt 的 8 个回答做归一化（减均值除标准差），天然限制了极端更新，去掉 KL 后，clip 约束仍在，模型获得了更大的探索空间。虽然 KL 系数为 0，但仍进行监控。**）

MMLU-Pro+4.2%、TheoremQA+10.1% 平均推理能力+7.5%，定理应用类推理上进步明显。

1. 为什么用 Qwen2.5-7B 做基座？为什么选择冷启动 SFT + RLPR 两阶段？为什么不用纯 SFT？

指令跟随能力稳定，SFT 阶段格式收敛快，降低了后续 RL 阶段因格式崩溃导致的不稳定风险。RL 训练需要模型先具备基本的推理格式能力，否则探索空间太大、收敛困难。SFT 本质是模仿学习，上限受限于训练数据的分布和质量，数据覆盖的推理模式有限，模型容易过拟合到训练数据的表面模式，且无法自主发现更优的推理路径（如自我反思、多路径验证）。

1. 8K CoT 从哪里来？数据类型包含哪些任务？

用 Few-shot Prompting + 教师模型蒸馏。具体来说，给 DeepSeek-R1 提供 3-shot 示例（格式要求），让它为种子问题生成完整的推理轨迹。这样保证输出格式统一、推理链质量高。覆盖五类任务——数学推理 30%、科学推理 25%、逻辑推理 20%、代码推理 15%、人文社科 10%。

1. 77K 高质量 RL 样本，筛选时推理复杂度评分是怎么做的？为什么阈值设为 ≥ 3 ？

1-4分。1分模型靠预训练知识就能答对，RL 没有优化空间。2分太开放或太简单，奖励信号弱、区分度差。3分数据最多，需要多步推理、有确定答案，既能让模型学到推理模式，又不会因过难导致奖励稀疏。4分题需要深度理解占比很低，7B 模型对过难题目的学习效果差，容易训崩。 ≥ 3 在“质量”和“数量”之间取得了平衡。

1. 10-gram 去重避免数据污染？去重对象是什么？

RL数据来自公开数据集，极易与评测集存在重叠或改写，10-gram比短5-gram更严格，能捕捉到片段级重复，防止“换皮题”导致的数据污染。去重对象：题目，不需要对答案或 CoT 去重。

1. 如果继续优化这个项目，你更想从哪边入手：

从 结果-based 概率奖励 扩展到 process-level 概率奖励。

全模态生成式推荐

给定用户最近最多 100 条历史行为，以及用户、商品稀疏特征、多模态 embedding，输出 Top-10 推荐 item。评估指标是加权 NDCG@10，其中 conversion 权重高于 click，所以模型不能只学“用户可能点什么”，还要显式区分曝光、点击、转化等行为价值。

官方 baseline 本质上是 SASRec / Transformer 序列编码：把用户历史编码成一个用户向量，再和候选 item 向量做 ANN 检索。它的问题是：第一，推荐目标仍然是向量相似度检索，不是直接生成目标 item；第二，原子 item ID 空间很大，直接建模泛化差。

我的整体采用 **协作式 SID + Decoder-only 生成推荐**。第一阶段先训练一个专用 Transformer，用用户历史、多模态特征、行为和时间特征，通过 InfoNCE 学到包含协同信号的 item embedding；再对这个 embedding 做两级 RQ-KMeans，构造唯一化 Semantic ID。第二阶段训练主推荐模型，把用户历史作为 prefix，自回归生成目标的 `action` → `sid1` → `sid2`。推理时通过合法 SID 约束解码、SID 反查 item、历史行为过滤，得到最终 Top-10。

1. 为什么要做生成式推荐，而不是沿用 baseline 的“用户向量 + ANN 检索”？

baseline 的做法更像传统召回模型：先编码用户兴趣，再从候选库里找相似 item。它的问题是训练目标和最终 Top-10 生成之间存在间接性，模型学的是向量空间相似度，而不是“下一个要推荐的离散 item 路径”。

生成式推荐把 item 离散成 Semantic ID 后，直接让模型生成目标 item 的 SID，相当于把推荐问题改写成 next-token prediction。这样有三个好处：第一，训练和推理目标更一致；第二，相似 item 可以共享前缀 SID，具备语义迁移能力；第三，可以像语言模型一样在生成过程中加入 action、时间、合法路径约束等条件。

1. 为什么不直接对官方多模态 embedding 做 K-Means 来构造 SID？

直接聚类多模态 embedding 只能表达内容相似，比如图片、文本语义相近。但推荐里的 item 相似不等于内容相似，还包含协同过滤信号：哪些 item 经常被相似用户连续交互，哪些 item 在点击或转化路径里扮演类似角色。

所以我先训练一个专用 Transformer，用 next item prediction + InfoNCE 学 item embedding。这个 embedding 同时吸收内容特征和行为共现信息，再做 RQ-KMeans 得到的 SID 更适合推荐任务。简单说：原始 embedding 是“内容语义”，协作式 embedding 是“推荐语义”。

1. 协作式 SID 是怎么构造的？

阶段 A 输入用户历史序列，每个 item token 包含 item sparse features、官方多模态 embedding、历史 action embedding 和时间特征。专用 Transformer 根据历史前缀预测下一个 item，用 InfoNCE 拉近真实 next item，推远负样本 item。

训练完成后取学习到的 item embedding 做两级 RQ-KMeans。第一级 K-Means 得到粗粒度 `sid1`；第二级不是直接再聚类原向量，而是对残差 $e_i - c_{\{sid1\}}$ 做 K-Means，得到细粒度 `sid2`。最后每个 item 被映射成 `(sid1, sid2)`。

1. InfoNCE 在 SID 构造阶段具体起什么作用？为什么大负样本库放在这里？

InfoNCE 的作用是学习可检索、可区分的 item 表示：给定用户历史表示 h_t ，真实 next item 是正样本，其他 item 是负样本，模型要让 $\text{sim}(h_t, e_{\text{pos}})$ 高于 $\text{sim}(h_t, e_{\text{neg}})$ 。

大负样本库适合放在 SID 构造阶段，因为这一阶段本质是 contrastive learning，负样本越丰富，item embedding 边界越清楚，后续 SID 质量越高。主推荐模型阶段已经变成生成式交叉熵训练，目标是生成 $\text{action}/\text{sid1}/\text{sid2}$ ，不再是向量检索式对比学习，所以不把大负样本库硬塞进主模型。

1. 为什么采用两层 SID，而不是一层或三层？

不用三层主要是比赛数据规模和生成稳定性的权衡。如果两层 codebook 都是 16K，组合空间是 $16384 \times 16384 \approx 2.68e8$ ，已经远大于决赛约 1700 万级 item 空间。三层虽然容量更大，但会增加 $\text{sid1} \rightarrow \text{sid2} \rightarrow \text{sid3}$ 的误差传播和 beam search 复杂度，当前瓶颈不是容量，而是 SID 质量、碰撞率和生成稳定性。

1. SID 碰撞怎么处理？为什么碰撞是严重问题？

标准 RQ-KMeans 可能让多个 item 映射到同一个 $(\text{sid1}, \text{sid2})$ 。解决方式是二级碰撞解决：先按最近质心分配 $(\text{sid1}, \text{sid2})$ ，然后检查冲突。如果多个 item 占用同一个 pair，就让冲突 item 在第二级码本里搜索下一个最近且未占用的中心，尽量保证 $\text{一个 item} \leftrightarrow \text{一个唯一 SID pair}$ 。

1. 主推荐模型为什么用 Decoder-only，而不是 Encoder-Decoder？

原版 OneRec 在工业场景里有复杂多路用户行为和超长序列，Encoder-Decoder 更自然。但 TAAC2025 用户历史最长只有 100，数据已经是结构化特征和 embedding，不需要很重的多路 Encoder。

1. 为什么输出顺序是 $\text{action} \rightarrow \text{sid1} \rightarrow \text{sid2}$ ，而不是只生成 SID？

因为 click 和 conversion 对应的 item 分布不一样，且评估里 conversion 权重更高。如果只生成 SID，模型可能把“用户可能点击的东西”和“用户可能转化的东西”混在同一个目标里。

先生成 action，相当于把目标行为作为条件，再生成对应的 SID： $P(a, s1, s2 | H) = P(a | H) \cdot P(s1 | H, a) \cdot P(s2 | H, a, s1)$ 。这样模型先判断下一步行为意图，再在这个行为条件下选择 item，能更好地区分曝光、点击、转化场景。

1. Action-conditioning 是怎么做的？和 action prediction 有什么区别？

action prediction 是目标端的任务，模型要预测下一个行为类型；action-conditioning 是输入端的条件调制，历史里每个 item 都根据它当时的行为类型改变表示。

具体可以用 Gated Fusion 或 FiLM。Gated Fusion 是让 action embedding 和 item 表示通过门控融合；FiLM 是由 action embedding 生成 γ/β ，对 item 表示做缩放和平移。这样同一个 item 在“曝光过”“点击过”“转化过”三种上下文下会有不同语义，而不是被当成同一种交互。

1. 时间特征为什么重要？用了哪些时间特征？

推荐里的兴趣有很强时效性：刚刚连续浏览的品类通常更能代表短期意图，广告/商品也有日周期、周周期和生命周期变化。如果没有时间特征，模型只能看到 item 顺序，看不到行为间隔和周期规律。

我保留两类最稳的时间特征：一是相对时间间隔 $\Delta t = t_i - t_{i-1}$ ，用 log-bucket 离散化后做 embedding；二是绝对时间特征，比如 hour-of-day、day-of-week。这样既能表达会话连续性，也能表达周期性。

1. 推理阶段怎么保证生成的 SID 是合法的？

离线构造 SID 后，可以得到一个 $sid1 \rightarrow \text{合法 } sid2 \text{ 集合}$ 的映射。推理生成 $sid2$ 时，只允许模型在当前 $sid1$ 对应的合法后缀里选择，避免生成训练集中不存在的 $(sid1, sid2)$ 。

1. 这个方案和 OneRec 原版有什么区别？

保留的是核心范式：把 item 离散成层级 SID，模型直接生成目标 SID，再映射回真实 item。

主要调整有三点。第一，原版 OneRec 面向快手工业短视频，有原始视频、caption、ASR、OCR、多帧图像和超长用户序列；本赛题只有官方多模态 embedding 和最长 100 的历史序列。第二，原版更偏 Encoder-Decoder 和复杂 reward system，我这里采用更适合比赛数据的 Decoder-only。第三，原版常用三层 SID，我这里用两层 SID，因为容量已经足够，并且生成更稳定。

1. 这个项目最容易被质疑的点是什么？怎么回答？

第一是“你是不是只是套 OneRec？”回答时要强调适配点：数据不是原始内容而是官方 embedding，历史长度较短，conversion 权重更高，所以我选择两层 SID、Decoder-only、action-first 生成和合法约束解码，而不是照搬工业 OneRec。

第二是“为什么 SID 一定更好？”可以说 SID 不是一定天然更好，它的收益依赖 SID 质量。如果直接用原始 embedding 聚类，效果可能一般；所以我先用 InfoNCE 学协作式 embedding，再量化成 SID，核心就是提高 SID 的推荐语义质量。

第三是“生成式会不会比 ANN 慢？”可以承认纯 beam search 的代价更高，所以这里用了两层 SID、合法后缀约束和较短输出序列，生成长度只有 3 个 token；如果上工业系统，也可以把生成候选和 ANN 召回结合。

1. 如果继续优化这个项目，你会从哪里入手？

第一，增强 SID 构造质量：尝试 LogQ debiasing、热度去偏和更大的 in-batch negatives，避免热门 item 主导聚类。

第二，引入混合召回：生成 SID 提供语义路径召回，同时保留 ANN 向量召回，两路候选融合后重排，兼顾精确匹配和语义泛化。

第三，做更细的 action/value 建模：conversion 权重更高，可以在 loss 权重、采样策略或目标 action 条件上加强高价值行为，而不是只把 click 和 conversion 当成普通分类标签。